In [1]:
```
import os
import numpy as np
import pandas as pd

import seaborn as sns
import plotly.express as px
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.metrics import euclidean_distances
from scipy.spatial.distance import cdist

import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
data= pd.read_csv('C:\Projects\Spotify_Music_Recommendation_Application\data\d
genre_data = pd.read_csv('C:\Projects\Spotify_Music_Recommendation_Applicatior
year_data = pd.read_csv('C:\Projects\Spotify_Music_Recommendation_Application\
```

In [3]:
```
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 170653 entries, 0 to 170652
Data columns (total 19 columns):
 #   Column            Non-Null Count   Dtype  
---  ------            --------------   -----  
 0   valence           170653 non-null  float64
 1   year              170653 non-null  int64  
 2   acousticness      170653 non-null  float64
 3   artists           170653 non-null  object 
 4   danceability      170653 non-null  float64
 5   duration_ms       170653 non-null  int64  
 6   energy            170653 non-null  float64
 7   explicit          170653 non-null  int64  
 8   id                170653 non-null  object 
 9   instrumentalness  170653 non-null  float64
 10  key               170653 non-null  int64  
 11  liveness          170653 non-null  float64
 12  loudness          170653 non-null  float64
 13  mode              170653 non-null  int64  
 14  name              170653 non-null  object 
 15  popularity        170653 non-null  int64  
 16  release_date      170653 non-null  object 
 17  speechiness       170653 non-null  float64
 18  tempo             170653 non-null  float64
dtypes: float64(9), int64(6), object(4)
memory usage: 24.7+ MB
None
```

In [4]:
```
print(genre_data.info())
```

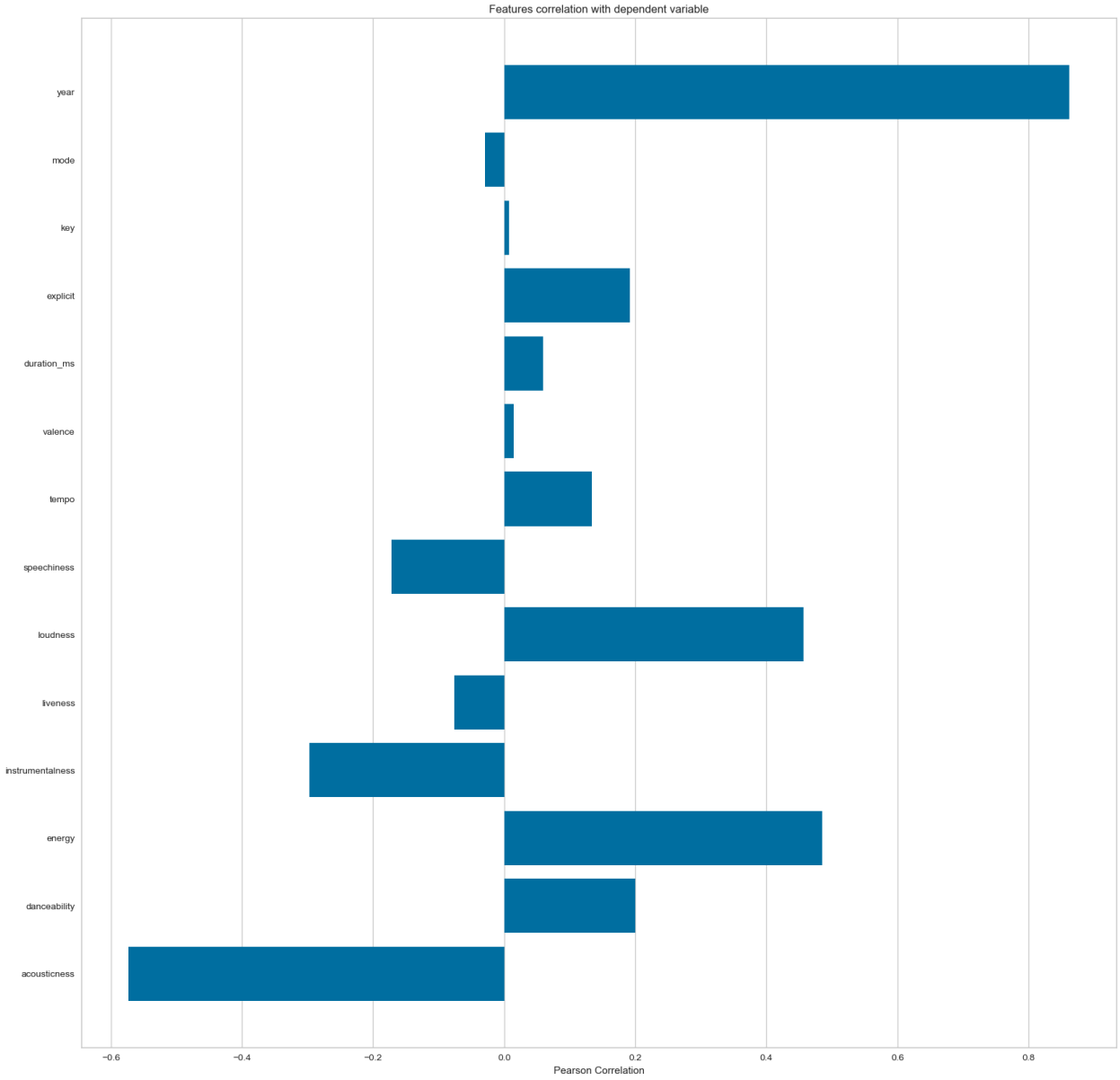
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2973 entries, 0 to 2972
Data columns (total 14 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   mode              2973 non-null   int64  
 1   genres            2973 non-null   object 
 2   acousticness      2973 non-null   float64
 3   danceability      2973 non-null   float64
 4   duration_ms       2973 non-null   float64
 5   energy            2973 non-null   float64
 6   instrumentalness  2973 non-null   float64
 7   liveness          2973 non-null   float64
 8   loudness          2973 non-null   float64
 9   speechiness       2973 non-null   float64
 10  tempo             2973 non-null   float64
 11  valence           2973 non-null   float64
 12  popularity        2973 non-null   float64
 13  key               2973 non-null   int64  
dtypes: float64(11), int64(2), object(1)
memory usage: 325.3+ KB
None
```

In [5]:
```
print(year_data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 14 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   mode              100 non-null    int64  
 1   year              100 non-null    int64  
 2   acousticness      100 non-null    float64
 3   danceability      100 non-null    float64
 4   duration_ms       100 non-null    float64
 5   energy            100 non-null    float64
 6   instrumentalness  100 non-null    float64
 7   liveness          100 non-null    float64
 8   loudness          100 non-null    float64
 9   speechiness       100 non-null    float64
 10  tempo             100 non-null    float64
 11  valence           100 non-null    float64
 12  popularity        100 non-null    float64
 13  key               100 non-null    int64  
dtypes: float64(11), int64(3)
memory usage: 11.1 KB
None
```

In [7]:
```
from yellowbrick.target import FeatureCorrelation

feature_names = ['acousticness', 'danceability', 'energy', 'instrumentalness',
       'liveness', 'loudness', 'speechiness', 'tempo', 'valence','duration_ms'
```

```
X, y = data[feature_names], data['popularity']

# Create a list of the feature names
features = np.array(feature_names)

# Instantiate the visualizer
visualizer = FeatureCorrelation(labels=features)

plt.rcParams['figure.figsize']=(20,20)
visualizer.fit(X, y)     # Fit the data to the visualizer
visualizer.show()
```



Out[7]: <Axes: title={'center': 'Features correlation with dependent variable'}, xlabel='Pearson Correlation'>

In [8]:
```
def get_decade(year):
    period_start = int(year/10) * 10
```

```
    decade = '{}s'.format(period_start)
    return decade

data['decade'] = data['year'].apply(get_decade)

sns.set(rc={'figure.figsize':(11 ,6)})
sns.countplot(data['decade'])
```

Out[8]:
```
<Axes: xlabel='count', ylabel='decade'>
```

In [9]:
```
sound_features = ['acousticness', 'danceability', 'energy', 'instrumentalness'
fig = px.line(year_data, x='year', y=sound_features)
fig.show()
```

In [10]:
```
top10_genres = genre_data.nlargest(10, 'popularity')

fig = px.bar(top10_genres, x='genres', y=['valence', 'energy', 'danceability',
fig.show()
```

Clustering Genres with K-Means

In [11]:
```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

cluster_pipeline = Pipeline([('scaler', StandardScaler()), ('kmeans', KMeans(r
X = genre_data.select_dtypes(np.number)
cluster_pipeline.fit(X)
genre_data['cluster'] = cluster_pipeline.predict(X)
```

In [12]:
```
# Visualizing the Clusters with t-SNE

from sklearn.manifold import TSNE

tsne_pipeline = Pipeline([('scaler', StandardScaler()), ('tsne', TSNE(n_compor
genre_embedding = tsne_pipeline.fit_transform(X)
projection = pd.DataFrame(columns=['x', 'y'], data=genre_embedding)
projection['genres'] = genre_data['genres']
projection['cluster'] = genre_data['cluster']

fig = px.scatter(
    projection, x='x', y='y', color='cluster', hover_data=['x', 'y', 'genres']
fig.show()
```

```
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 2973 samples in 0.013s...
[t-SNE] Computed neighbors for 2973 samples in 0.210s...
[t-SNE] Computed conditional probabilities for sample 1000 / 2973
[t-SNE] Computed conditional probabilities for sample 2000 / 2973
[t-SNE] Computed conditional probabilities for sample 2973 / 2973
[t-SNE] Mean sigma: 0.777516
[t-SNE] KL divergence after 250 iterations with early exaggeration: 76.102356
[t-SNE] KL divergence after 1000 iterations: 1.392492
```

In [13]:
```
#Clustering of Song Data
song_cluster_pipeline = Pipeline([('scaler', StandardScaler()),('kmeans', KMea
X = data.select_dtypes(np.number)
number_cols = list(X.columns)
song_cluster_pipeline.fit(X)
song_cluster_labels = song_cluster_pipeline.predict(X)
data['cluster_label'] = song_cluster_labels
```

In [14]:
```
# Visualizing the Clusters with PCA

from sklearn.decomposition import PCA

pca_pipeline = Pipeline([('scaler', StandardScaler()), ('PCA', PCA(n_component
song_embedding = pca_pipeline.fit_transform(X)
projection = pd.DataFrame(columns=['x', 'y'], data=song_embedding)
projection['title'] = data['name']
projection['cluster'] = data['cluster_label']

fig = px.scatter(
    projection, x='x', y='y', color='cluster', hover_data=['x', 'y', 'title'])
fig.show()
```

Build Recommender System

Based on the analysis and visualizations, it’s clear that similar genres tend to have data points that are located close to each other while similar types of songs are also clustered together.

This observation makes perfect sense. Similar genres will sound similar and will come from similar time periods while the same can be said for songs within those genres. We can use this idea to build a recommendation system by taking the data points of the songs a user has listened to and recommending songs corresponding to nearby data points.

Spotipy is a Python client for the Spotify Web API that makes it easy for developers to fetch data and query Spotify’s catalog for songs. We have to install using pip install spotipy

We will need to create an app on the Spotify Developer’s page and save your Client

ID and secret key.

In [15]:
```
!pip install spotipy
```

```
Collecting spotipy
  Downloading spotipy-2.25.0-py3-none-any.whl.metadata (4.7 kB)
Collecting redis>=3.5.3 (from spotipy)
  Downloading redis-5.2.1-py3-none-any.whl.metadata (9.1 kB)
Requirement already satisfied: requests>=2.25.0 in c:\users\rushx\anaconda3\li
b\site-packages (from spotipy) (2.32.3)
Requirement already satisfied: urllib3>=1.26.0 in c:\users\rushx\anaconda3\lib\
site-packages (from spotipy) (2.2.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\rushx\anaco
nda3\lib\site-packages (from requests>=2.25.0->spotipy) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\rushx\anaconda3\lib\sit
e-packages (from requests>=2.25.0->spotipy) (3.7)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\rushx\anaconda3\l
ib\site-packages (from requests>=2.25.0->spotipy) (2024.8.30)
Downloading spotipy-2.25.0-py3-none-any.whl (30 kB)
Downloading redis-5.2.1-py3-none-any.whl (261 kB)
Installing collected packages: redis, spotipy
Successfully installed redis-5.2.1 spotipy-2.25.0
```

In [16]:
```
import spotipy
from spotipy.oauth2 import SpotifyClientCredentials
from collections import defaultdict
import os

# Set your Spotify credentials
os.environ["SPOTIFY_CLIENT_ID"] = "70e8bce0cba74248a5f501188e45485c"
os.environ["SPOTIFY_CLIENT_SECRET"] = "8d8633b4e6f648548040a12fd7f995c7"

sp = spotipy.Spotify(auth_manager=SpotifyClientCredentials(client_id=os.enviro

def find_song(name, year):
    song_data = defaultdict()
    results = sp.search(q= 'track: {} year: {}'.format(name,year), limit=1)
    if results['tracks']['items'] == []:
        return None

    results = results['tracks']['items'][0]
    track_id = results['id']
    audio_features = sp.audio_features(track_id)[0]

    song_data['name'] = [name]
    song_data['year'] = [year]
    song_data['explicit'] = [int(results['explicit'])]
    song_data['duration_ms'] = [results['duration_ms']]
    song_data['popularity'] = [results['popularity']]

    for key, value in audio_features.items():
        song_data[key] = value

    return pd.DataFrame(song_data)
```

In [17]:

```
from collections import defaultdict
from sklearn.metrics import euclidean_distances
from scipy.spatial.distance import cdist
import difflib

number_cols = ['valence', 'year', 'acousticness', 'danceability', 'duration_ms
 'instrumentalness', 'key', 'liveness', 'loudness', 'mode', 'popularity', 'spe


def get_song_data(song, spotify_data):

    try:
        song_data = spotify_data[(spotify_data['name'] == song['name'])
                                 & (spotify_data['year'] == song['year'])].iloc
        return song_data

    except IndexError:
        return find_song(song['name'], song['year'])


def get_mean_vector(song_list, spotify_data):

    song_vectors = []

    for song in song_list:
        song_data = get_song_data(song, spotify_data)
        if song_data is None:
            print('Warning: {} does not exist in Spotify or in database'.forma
            continue
        song_vector = song_data[number_cols].values
        song_vectors.append(song_vector)

    song_matrix = np.array(list(song_vectors))
    return np.mean(song_matrix, axis=0)


def flatten_dict_list(dict_list):

    flattened_dict = defaultdict()
    for key in dict_list[0].keys():
        flattened_dict[key] = []

    for dictionary in dict_list:
        for key, value in dictionary.items():
            flattened_dict[key].append(value)

    return flattened_dict


def recommend_songs( song_list, spotify_data, n_songs=10):

    metadata_cols = ['name', 'year', 'artists']
    song_dict = flatten_dict_list(song_list)
```

```
    song_center = get_mean_vector(song_list, spotify_data)
    scaler = song_cluster_pipeline.steps[0][1]
    scaled_data = scaler.transform(spotify_data[number_cols])
    scaled_song_center = scaler.transform(song_center.reshape(1, -1))
    distances = cdist(scaled_song_center, scaled_data, 'cosine')
    index = list(np.argsort(distances)[:, :n_songs][0])

    rec_songs = spotify_data.iloc[index]
    rec_songs = rec_songs[~rec_songs['name'].isin(song_dict['name'])]
    return rec_songs[metadata_cols].to_dict(orient='records')
```

In [18]:
```
import time

def get_song_data(song, spotify_data):
    try:
        song_data = spotify_data[(spotify_data['name'] == song['name']) & (spo
    except IndexError:
        try:
            # Retry the request after a brief delay
            time.sleep(2)
            song_data = spotify_data[(spotify_data['name'] == song['name']) &
        except IndexError:
            print(f"Error: Could not find data for song '{song['name']}' ({sor
            song_data = None
    return song_data
```

In [19]:
```
recommend_songs([
                {'name': 'Antidote', 'year': 2015},
                {'name': 'See You Again (feat. Charlie Puth)', 'year': 2015}
                ],  data)
```

Out[19]:
```
[{'name': 'Spicy (feat. Post Malone)',
  'year': 2020,
  'artists': "['Ty Dolla $ign', 'Post Malone']"},
 {'name': 'do re mi', 'year': 2017, 'artists': "['blackbear']"},
 {'name': 'Topanga', 'year': 2018, 'artists': "['Trippie Redd']"},
 {'name': 'Modern Loneliness', 'year': 2020, 'artists': "['Lauv']"},
 {'name': '...And To Those I Love, Thanks For Sticking Around',
  'year': 2020,
  'artists': "['$uicideBoy$']"},
 {'name': "Ballin' (with Roddy Ricch)",
  'year': 2019,
  'artists': "['Mustard', 'Roddy Ricch']"},
 {'name': 'Dead Eyes',
  'year': 2019,
  'artists': "['Promoting Sounds', 'Powfu', 'Ouse']"},
 {'name': 'GREECE (feat. Drake)',
  'year': 2020,
  'artists': "['DJ Khaled', 'Drake']"},
 {'name': 'Go Flex', 'year': 2016, 'artists': "['Post Malone']"},
 {'name': 'Pray For Me (with Kendrick Lamar)',
  'year': 2018,
  'artists': "['The Weeknd', 'Kendrick Lamar']"}]
```

In [20]:
```
recommend_songs([
                 {'name': 'Come As You Are', 'year':1991},
                {'name': 'Smells Like Teen Spirit', 'year': 1991},
                {'name': 'Lithium', 'year': 1992},
                {'name': 'All Apologies', 'year': 1993},
                 ],  data)
```

Out[20]:
```
[{'name': 'Hanging By A Moment', 'year': 2000, 'artists': "['Lifehouse']"},
 {'name': 'Kiss Me', 'year': 1997, 'artists': "['Sixpence None The Riche
r']"},
 {'name': "Breakfast At Tiffany's",
  'year': 1995,
  'artists': "['Deep Blue Something']"},
 {'name': 'Otherside', 'year': 1999, 'artists': "['Red Hot Chili Peppers']"},
 {'name': "It's Not Living (If It's Not With You)",
  'year': 2018,
  'artists': "['The 1975']"},
 {'name': 'No Excuses', 'year': 1994, 'artists': "['Alice In Chains']"},
 {'name': 'Wherever You Will Go', 'year': 2001, 'artists': "['The Callin
g']"},
 {'name': 'Ballbreaker', 'year': 1995, 'artists': "['AC/DC']"},
 {'name': 'Runaway (U & I)', 'year': 2015, 'artists': "['Galantis']"},
 {'name': "Club Can't Handle Me (feat. David Guetta)",
  'year': 2010,
  'artists': "['Flo Rida', 'David Guetta']"}]
```